

TRANSACTIQ

Intelligent Payment Processing, Reconciliation & Settlement Platform

A simple explanation of a serious fintech engineering project
Designed so that a beginner, a recruiter, a lecturer, or a technical person
can understand what the system does, why it exists, and how it works.

Core idea: TransactIQ is a software platform that helps businesses receive, track, verify, reconcile and settle digital payments while keeping a clear and secure record of what happened.

Project Concept Document

Version 1.0 • September 2026

1. What is TransactIQ?

TransactIQ is a proposed financial technology (fintech) platform. Its job is to help a business handle digital payment transactions in an organised and trustworthy way. A transaction is simply a record of money moving from one place to another—for example, a customer paying ₦5,000 for a product.

The system does not have to move real money during development. For the portfolio version, payment providers can be simulated safely. The important goal is to reproduce the engineering problems that real payment platforms must solve: avoiding duplicate charges, tracking transaction states, handling failed or delayed payments, notifying merchants, reconciling records, keeping audit trails, and preparing correct settlement information.

In one sentence: TransactIQ is like a very careful digital accountant and traffic controller for payments: it receives payment requests, follows each transaction, records every important event, checks that the records agree, and helps determine what money should finally be settled to a merchant.

2. The problem TransactIQ is solving

Problem	Simple example	What TransactIQ does
Duplicate transactions	A customer taps Pay twice because the first response was slow.	Uses idempotency so one request does not accidentally create two payments.
Unknown transaction status	A payment appears to be processing and nobody knows whether it eventually succeeded.	Uses clear transaction states and status history.
Different records	The merchant says 100 payments happened, while another record shows 98.	Runs reconciliation and highlights differences.
Failed notifications	A payment succeeds but the merchant's server never receives the notification.	Uses webhooks, logs and controlled retries.
Poor traceability	Someone changes an important financial record and nobody knows who did it.	Keeps an audit trail of important actions.
Suspicious activity	One account suddenly makes many unusual payments in a short time.	Uses rules and later AI/ML to flag anomalies.

3. The main goal

The main goal is to build a secure, reliable and scalable payment-processing system that demonstrates how modern fintech software can manage the complete journey of a transaction—from the moment a merchant requests a payment to the point where the transaction is verified, recorded, reconciled and included in settlement.

Imagine this simple story

A shop sells a phone for ₦100,000. The customer wants to pay electronically. The shop sends a payment request to TransactIQ. TransactIQ gives the request a unique reference and records it. A

payment provider processes the request. The result may be successful, failed or still pending. TransactIQ records the result. If successful, the merchant can later see the transaction, receive a webhook notification, and have the transaction included in settlement. If another record disagrees with TransactIQ, the reconciliation engine finds the difference for investigation.

4. Major users of the system

Merchant: A business that uses TransactIQ to receive and monitor payments.

Customer: The person who makes a payment. The customer may interact with a payment page or an external provider.

Administrator: Manages merchants, configurations, users, permissions and platform operations.

Auditor: Reviews transactions, reconciliation exceptions, settlements and audit records.

Operations Officer: Investigates failed transactions, disputes, webhook problems and reconciliation exceptions.

5. Core features

Merchant Management: Register merchants, create accounts, manage API credentials and configure payment settings.

Payment API: Allow approved merchant applications to create and check payment transactions through secure APIs.

Transaction Engine: Create unique transaction references and control valid transaction state changes.

Idempotency: Prevent the same payment request from accidentally creating duplicate transactions.

Payment Provider Simulation: Simulate successful, failed and pending responses so the platform can be tested without real money.

Webhooks: Notify merchant systems when important events happen, with signing, logs and retry handling.

Ledger: Keep a structured financial record of debits, credits and merchant balances.

Reconciliation: Compare internal records with external/provider records and identify mismatches.

Settlement: Group eligible successful transactions and calculate what should be paid to merchants.

Dispute Management: Record and track transaction complaints or exceptions.

Audit Trail: Record important actions: who did what, when, and what changed.

Fraud/Anomaly Rules: Flag unusual behaviour for review; later this can be enhanced with machine learning.

Reports & Analytics: Show payment volume, success rate, failures, pending transactions, settlement totals and exceptions.

Role-Based Access Control: Give each user only the permissions needed for their job.

Notifications: Provide controlled email/SMS/in-app notifications for selected events in later versions.

6. The technical workflow — from payment request to settlement

The workflow below explains how the different parts of the application communicate. It is deliberately written in simple language first, followed by the technical interpretation.

Step 1 — Merchant starts a payment

A merchant's application sends a payment request with amount, customer details and a unique reference.

Step 2 — API authentication

The API checks the merchant's credentials/API key and confirms that the merchant is allowed to perform the operation.

Step 3 — Validation

The system checks that required information is present, the amount is valid and the request follows the rules.

Step 4 — Idempotency check

The system checks whether the same idempotency key was already processed. If yes, it returns the existing result instead of creating a duplicate.

Step 5 — Transaction creation

A transaction record is created with a unique ID and an initial state such as INITIATED.

Step 6 — Queue/background processing

Long-running or retryable work can be placed on a queue so the API remains responsive.

Step 7 — Payment provider

TransactIQ sends the payment request to a simulated or test payment provider.

Step 8 — Provider response

The provider returns SUCCESS, FAILED or PENDING. The system records the response and timestamps.

Step 9 — Ledger recording

For successful financial events, the ledger records the corresponding debit/credit entries inside a database transaction.

Step 10 — Webhook

The merchant receives a signed event notification such as payment.success. If delivery fails, the webhook system retries according to its policy.

Step 11 — Reconciliation

At scheduled intervals, internal transactions are compared with provider records. Exceptions are created for mismatches.

Step 12 — Settlement

Eligible successful transactions are grouped into a settlement batch, fees are calculated, and the amount payable to the merchant is produced.

Step 13 — Audit & reporting

Important actions and results remain available for authorised users through reports, logs and dashboards.

7. The system architecture

The project should be built as several cooperating layers. Think of a school: the front office receives people, teachers perform their work, the records office stores information, and management checks reports. Each software layer has a clear responsibility.

Layer	Technology / Component	Purpose
Frontend	React + TypeScript	Merchant/admin dashboards and user interfaces.
API	Laravel / PHP	Receives requests, authenticates users and exposes business APIs.
Database	PostgreSQL	Stores merchants, users, transactions, ledger entries, settlements and audit records.
Cache / Queue	Redis	Fast temporary data, rate limiting and background jobs.
Webhooks	Laravel Queue + HTTP	Sends event notifications to merchant systems and retries failures.
AI/Analytics service	Python + FastAPI (later phase)	Anomaly detection, analytics and intelligent transaction insights.
Containers	Docker	Packages the application so it runs consistently.
CI/CD	GitHub Actions	Automatically tests and deploys approved changes.
Cloud	AWS	Hosts the application and supporting services.
Monitoring	CloudWatch / application logs	Tracks errors, health, performance and operational events.

8. Simple technical picture

```
Merchant App → React Dashboard / API Client → Laravel API → Authentication & Validation → Transaction Service → Queue/Redis → Payment Provider → Transaction Result → Ledger → Webhook → Reconciliation → Settlement → Reports
```

In production-style thinking, the important point is that money-related operations should be carefully controlled. Database transactions, unique constraints, idempotency, access control, audit logs and tests are not optional decorations; they are part of the reliability design.

9. Important database entities

users — People who can sign in to the platform.

merchants — Businesses using TransactIQ.

api_keys — Credentials used by merchant applications to call the API.

customers — Customers associated with payment records where appropriate.

transactions — The main record for each payment attempt.

transaction_events — History of important status changes and provider events.

payment_attempts — Individual attempts to process a transaction.

ledger_accounts — Accounts used to represent financial positions.

ledger_entries — Debit/credit records forming the financial ledger.

webhook_deliveries — Records of webhook attempts, responses and retries.

reconciliation_runs — A record of each reconciliation process.

reconciliation_exceptions — Mismatches requiring investigation.

settlements — Settlement batches and their status.

audit_logs — Security and operational history of important actions.

10. Example transaction journey

Suppose a merchant wants to collect ₦50,000 from a customer. Here is what should happen:

1. The merchant sends a secure API request for ₦50,000.
2. TransactIQ checks the API key and validates the request.
3. TransactIQ checks the idempotency key.
4. The transaction is created with a unique reference such as TXN-20260914-000001.
5. The transaction enters PROCESSING.
6. The test payment provider returns SUCCESS.
7. TransactIQ records the successful event.
8. The ledger records the financial entries.
9. A signed payment.success webhook is sent to the merchant.
10. The transaction later appears in the reconciliation process.
11. If the provider's record agrees, it is marked MATCHED.
12. When eligible, the transaction becomes part of a settlement batch.

11. Transaction state machine

```
INITIATED → PROCESSING → SUCCESS  
INITIATED → PROCESSING → FAILED  
INITIATED → PROCESSING → PENDING → SUCCESS  
INITIATED → PROCESSING → PENDING → FAILED
```

The application must prevent invalid changes. For example, a completed SUCCESS transaction should not simply be changed back to PROCESSING. If a later correction is needed, it should be represented through a controlled process and recorded in the audit trail.

12. Security requirements

- Secure password hashing and authentication.
- Role-based access control and least-privilege permissions.
- API authentication and secret-key protection.
- HTTPS in deployment.
- Request validation and safe database queries.
- Rate limiting for sensitive endpoints.
- Webhook signatures so merchants can verify event authenticity.
- Protection against common web vulnerabilities such as SQL injection, XSS and CSRF.
- Audit logs for sensitive actions.
- Secrets stored outside source code.
- Careful handling of personally identifiable information.

- Automated security and functional tests before deployment.

13. How the project builds your job-market skills

Skill	How TransactIQ forces you to practise it
Laravel/PHP	Business logic, authentication, APIs, queues and database transactions.
React/TypeScript	Dashboards, forms, API communication and typed frontend code.
PostgreSQL	Relational modelling, indexes, constraints and transaction consistency.
Redis	Queues, caching, rate limiting and temporary state.
REST APIs	Merchant integration and system-to-system communication.
Fintech concepts	Transactions, ledgers, reconciliation, settlement and disputes.
Cybersecurity	Authentication, authorisation, secrets, signatures and audit trails.
Docker	Repeatable development and deployment environments.
CI/CD	Automated tests and deployment pipelines.
AWS	Cloud hosting, networking, storage, monitoring and scaling.
Python	A separate service for analytics/anomaly detection.
AI/ML	Later-stage anomaly detection and intelligent transaction insights.
System design	Queues, services, reliability, scalability and failure handling.
Testing	Unit, feature, API, integration and failure-path tests.

14. Development roadmap

Phase 1 — Foundation: Project setup, authentication, users, merchants, database design and basic dashboard.

Phase 2 — Payment API: API keys, payment creation, validation, transaction references and transaction states.

Phase 3 — Reliability: Idempotency, database transactions, queues, retries and error handling.

Phase 4 — Webhooks: Signed events, delivery logs, retry mechanism and merchant webhook testing.

Phase 5 — Ledger & Settlement: Double-entry ledger model, balances, fees, settlement batches and reports.

Phase 6 — Reconciliation: Import provider records, compare them with internal records and manage exceptions.

Phase 7 — Security & Audit: RBAC, rate limiting, audit logs, secure secrets and security testing.

Phase 8 — Intelligence: Rule-based anomaly detection followed by a Python/ML service.

Phase 9 — Cloud & DevOps: Docker, AWS, CI/CD, infrastructure automation, monitoring and load testing.

15. What makes TransactIQ a strong portfolio project?

The strength of TransactIQ is not the number of screens. Its strength is the engineering behind the screens. A good implementation shows that you understand what happens when networks fail, requests are repeated, providers return different statuses, webhooks do not arrive, records disagree, users have different permissions, and financial data must remain traceable.

Portfolio positioning: Instead of presenting it as 'a payment website', present it as a payment infrastructure and financial operations platform. That communicates backend engineering, fintech understanding, cloud readiness, security awareness and system-design ability.

16. What TransactIQ is NOT

TransactIQ should not be described as a direct clone of any existing Nigerian payment company. It is a learning and portfolio project inspired by common payment-infrastructure problems. During development, use simulated/test payment providers and never handle real customer money unless the required legal, regulatory and security requirements have been properly addressed.

17. Final vision

The long-term vision is for TransactIQ to become a small but technically serious demonstration of how a modern fintech platform can process transactions safely, communicate with other systems, keep reliable financial records, detect exceptions, reconcile data and prepare settlements.

Simple explanation for a primary-school student:

Imagine a very careful shopkeeper whose job is to watch every payment. When someone pays, the shopkeeper writes down exactly what happened. If someone tries to pay twice, the shopkeeper notices. If one notebook says 10 people paid but another says 9, the shopkeeper finds the missing record. At the end of the day, the shopkeeper calculates how much each business should receive. TransactIQ is the computer version of that very careful shopkeeper—but it can work with many businesses and thousands of transactions.

18. One-line project definition

TransactIQ is a secure, cloud-ready payment infrastructure platform that processes transactions, prevents duplicates, tracks payment events, notifies merchants, reconciles records and prepares settlements.